REVIEWER WALKTHROUGH

Invoice Approval: approval limits by role

One feature, taken from specification to a reviewed pull request, with the evidence around it.

Python · FastAPI | Spec Kit | Claude Code | pytest | mutmut | Playwright

Reviewer edition · Version 1.0 · 30 October 2026

OVERVIEW

The assignment, and what was delivered

The ask

Add **role-based approval limits** to a small invoice approval service, and take it from specification to a reviewed pull request with the evidence around it: tests, mutation scores, a browser test, a review with a verdict on every finding, and a log of every AI interaction.

The feature

A **Manager** may approve up to 50,000.00. A **Director** may approve up to 500,000.00. Above that, **two different approvers** are needed and the second must be a Director. A refused attempt returns a **clear reason** and changes **nothing**.

The outcome

177 unit tests and **1** browser test pass. Mutation score rose from **63.9%** to **100%**. **11** review findings each have a verdict. Every AI interaction is logged with **27** fixed keys.

Stack: Python 3.13 · FastAPI · pytest · mutmut 3.8 · Playwright · Spec Kit · Claude Code (Sonnet 5.5)

The original demo is ASP.NET Core. With no .NET SDK available and Python as the requirement, the backend was ported to FastAPI with the same layering, endpoints and web UI, and the 7 original tests were kept green before any feature work.

OVERVIEW

Results at a glance

177

Backend tests pass

63 policy, 62 service, 36 API, 16 repository contract

100%

Mutation score

132 of 132 mutants killed; the baseline was 63.9% (23 of 36)

1

Browser test passes

Manager is refused on an over-limit invoice, the reason is shown, the invoice stays pending

11

Review findings

6 fixed, 3 dismissed with a reason, 2 deferred and tracked

27

Keys on every log line

Prompt, tools, sub-agents, AI suggestion, human decision, verification, status

5

Project agents

spec-writer, implementer, test-reviewer, pr-reviewer, mutation-analyst

01

The feature

What the rules are, the decisions behind them, and the life of an invoice.

THE FEATURE

Who may approve what



Limits are inclusive. The tests try one cent either side of both limits.

	Up to 50,000.00	50,000.01 to 500,000.00	Above 500,000.00
Manager	approves	refused: needs a Director	may be the FIRST approver only
Director	approves	approves	first or second approver
Admin	never approves	never approves	never approves

Above 500,000.00 the same user can never count twice (names compared after trim and ignoring case), and a refused attempt changes nothing.

THE FEATURE

Decisions made, and why

1 Limits are inclusive

50,000.00 is allowed for a Manager; 50,000.01 needs a Director. Same at 500,000. Tests try one cent either side.

2 First approver above 500,000

A Manager or a Director. Only the second approver is constrained: it must be a Director.

3 "The same user"

The same account after trimming spaces and ignoring case, so `Dir.One` and `dir.one` are one person.

4 Admin cannot approve

Admin keeps reject and delete, never approval. The old Approver role became Manager.

5 A refusal is not a Reject

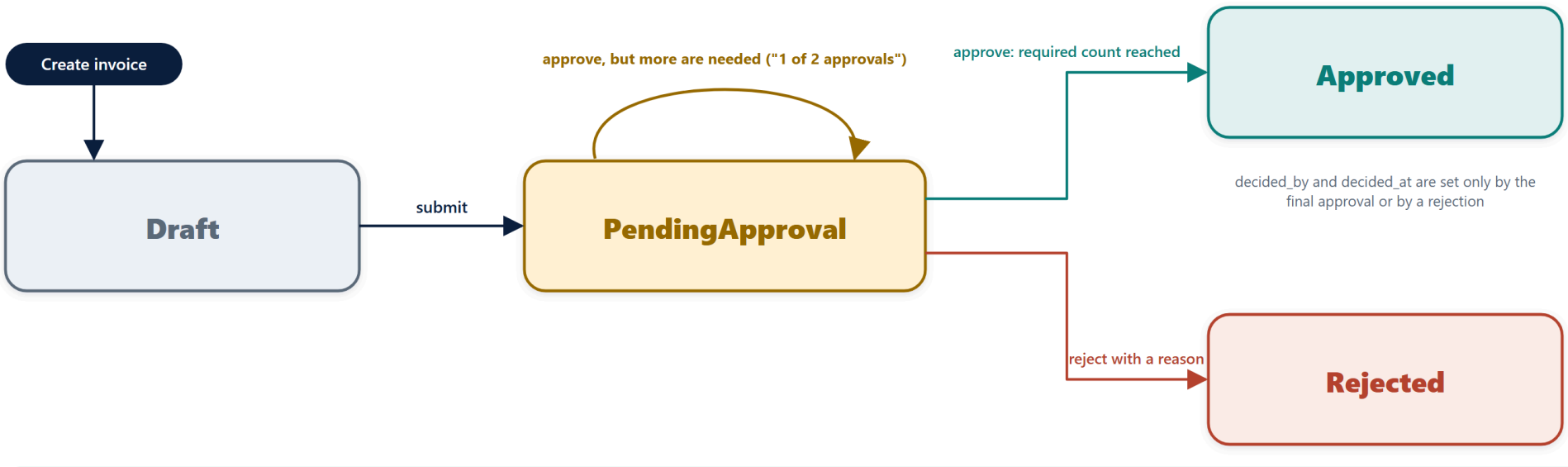
"Rejected attempt" means an approval the rules refuse: 403 with a reason, nothing written. The Reject button is unchanged.

6 Nothing changes on refusal

Every check runs before the first change, so there is no half-done state to clean up.

THE FEATURE

The life of an invoice



A refused approval (403) is not a transition: the invoice stays in the same state with the same approvals.

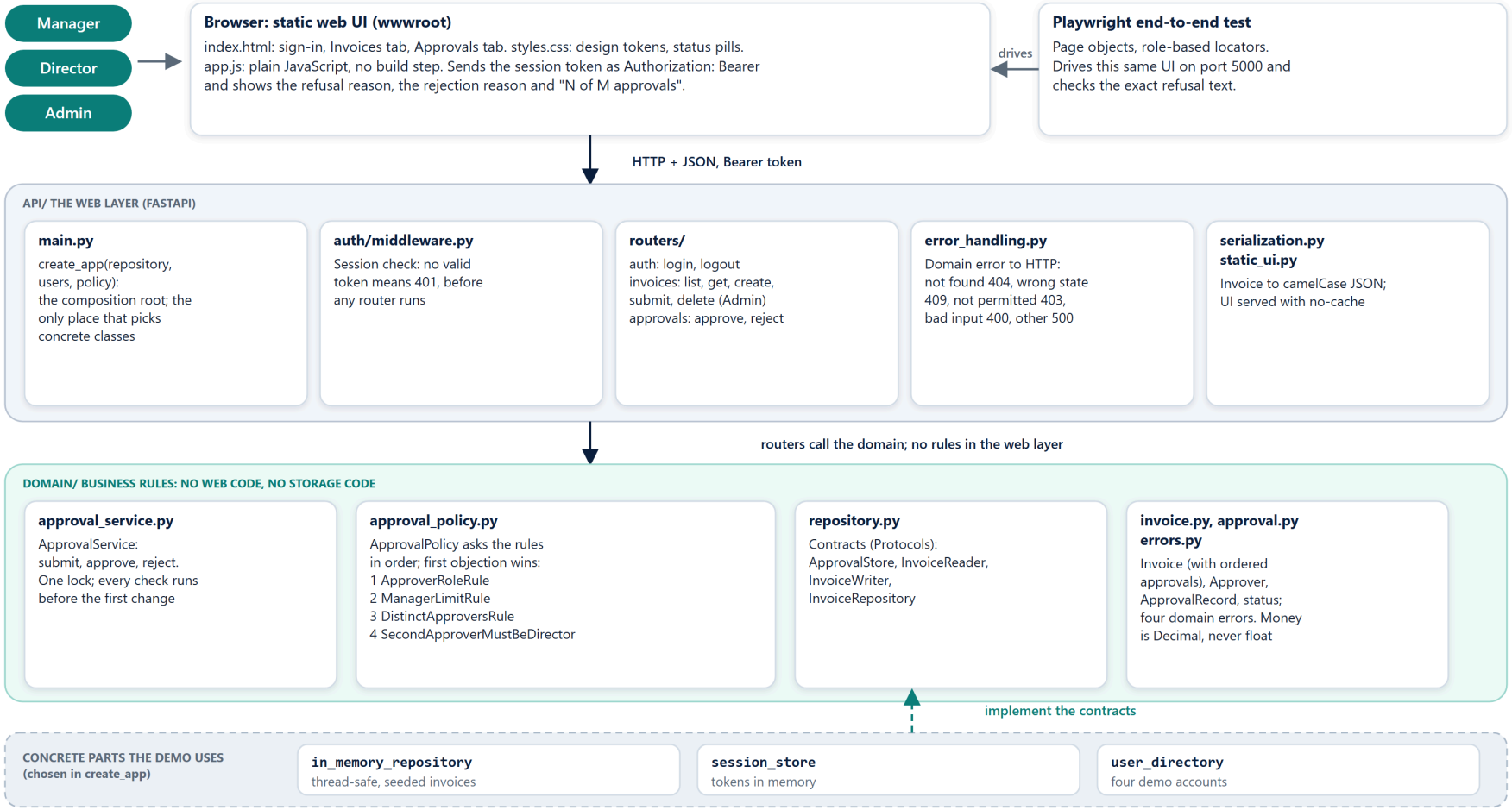
Wrong state (for example approving an already Approved invoice) is a 409; an unknown id is a 404. Admin can also delete an invoice.

02

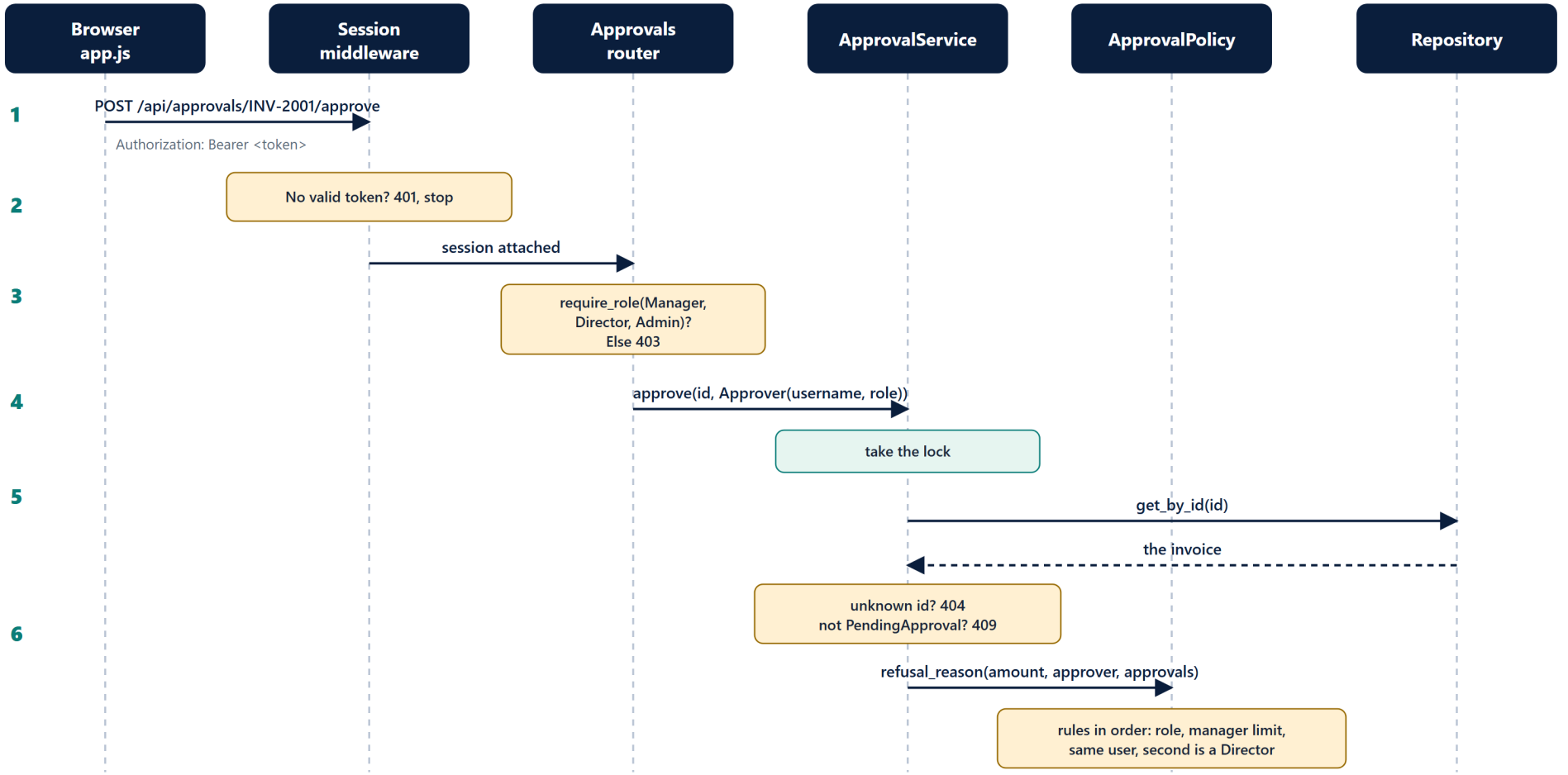
Architecture

Where each responsibility lives, how one request flows, and the design principles behind it.

System architecture

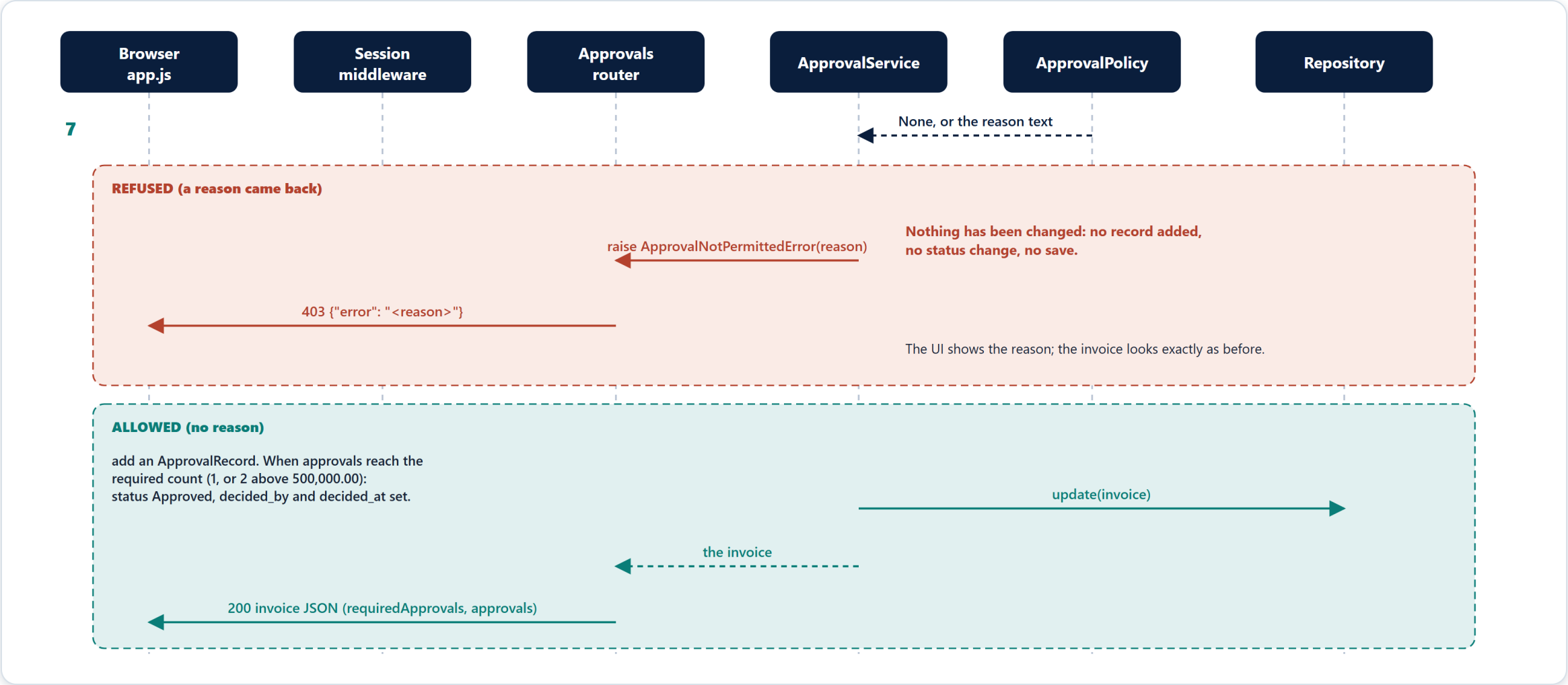


One approval request: the checks



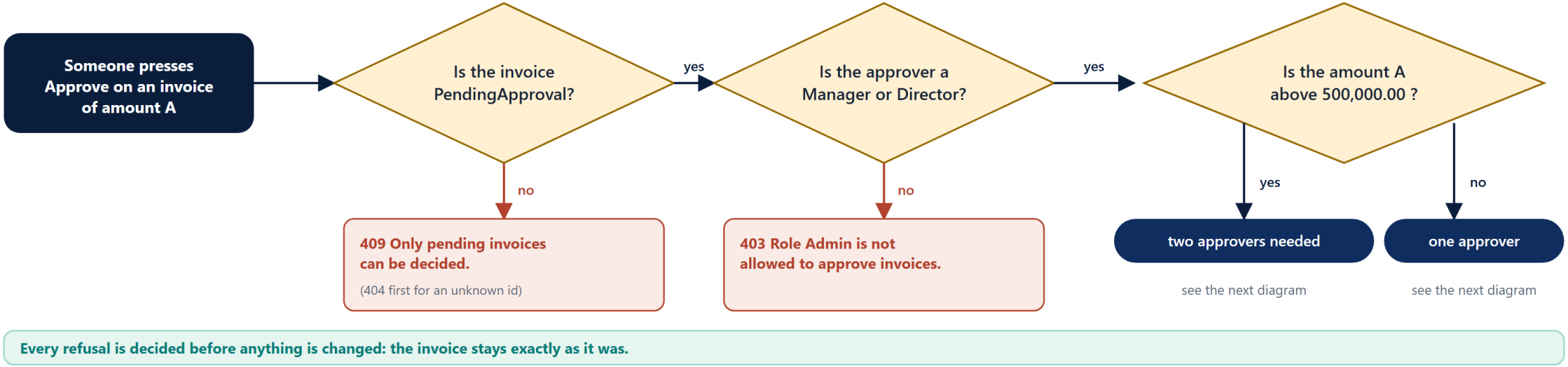
- The session is checked first (401), then the role (403), then the service takes a lock, loads the invoice (404, 409) and asks the policy.

One approval request: refused or allowed



ARCHITECTURE

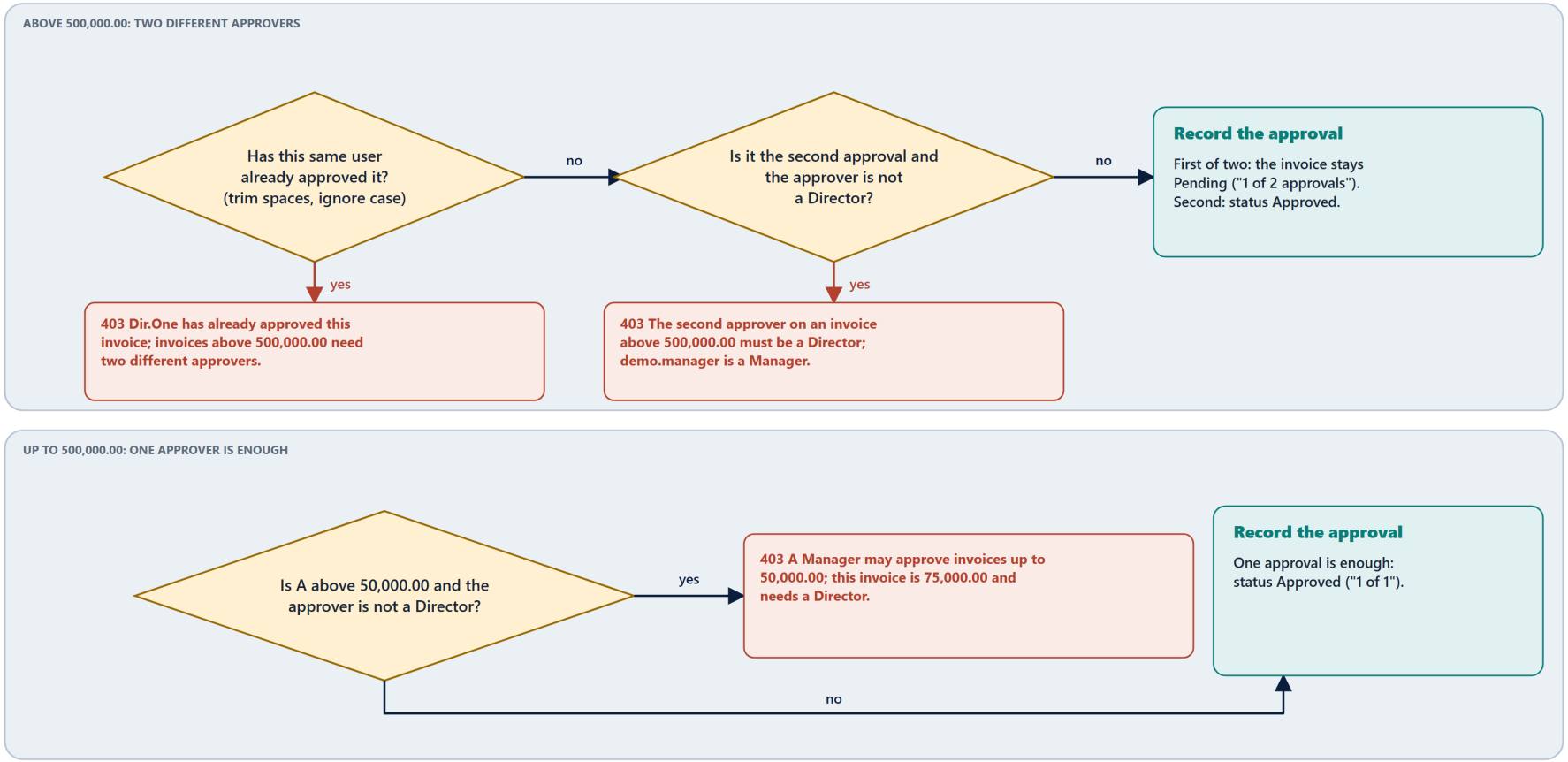
The approval rules: three gates



- Rules run in a **fixed order**: state, role, Manager limit, same user, second must be a Director.
- The **first objection wins** and its text is the reason shown to the user.
- Each red box is a real message from the code and is asserted word for word in a test.

ARCHITECTURE

The approval rules: one approver or two



Rule order: role, Manager limit, "same user", "second must be a Director". The first objection wins and its text is the reason shown.

Limits are inclusive: 50,000.00 and 500,000.00 are inside their band; one cent more moves to the next band.

ARCHITECTURE

Design: the SOLID principles, with proof

Principle	What changed	Proof
Single responsibility	main.py was split into wiring, error mapping and the session check; each approval rule is its own class.	Each file has one job; every rule class has its own tests.
Open / closed	Four rule classes plus an ApprovalPolicy that asks them in order; limits are data. A new rule is a new class.	`test_a_new_rule_plugs_into_the_policy_without_changing_the_existing_rules`
Liskov substitution	One contract test suite runs against the fake repository and the real one.	`test_repository_contract.py` (16 tests)
Interface segregation	The approval service needs two operations, so it takes an ApprovalStore, not the six-method repository.	`test_the_service_works_with_a_store_that_offers_only_find_and_save`
Dependency inversion	The service takes a policy and a clock as arguments; login uses a UserDirectory; create_app accepts replacements.	Tests inject a stub policy, a fixed clock and a one-user directory.

Behaviour did not change: all 134 earlier tests passed unchanged, 43 were added, and mutation stayed at 100%.

ARCHITECTURE

Key design decisions

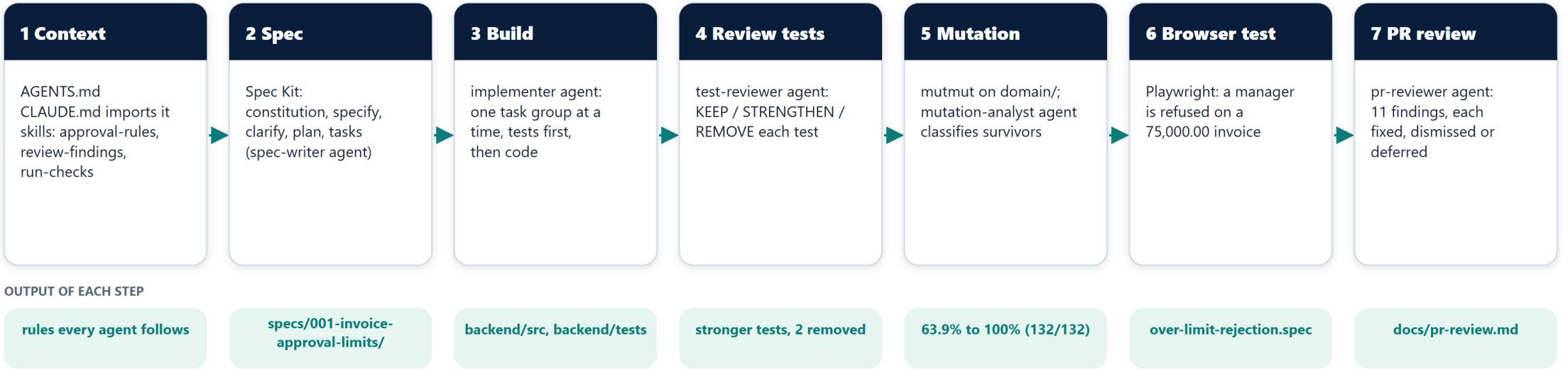
Decision	Choice	Why
Money	`Decimal`, never float	Exact comparison at 50,000.00 and 50,000.01.
Refusal signal	`ApprovalNotPermittedError` mapped to 403	Distinct from 409 (wrong state), 404 and 400 (bad input).
No partial writes	Check first, then change	The in-memory repository returns the live object, so changing first would leak state.
Atomic approvals	One lock in `ApprovalService`	Two clicks by one user could both pass the duplicate check (finding F1).
Same user	`strip().casefold()`	Stops `Dir.One` versus `dir.one` slipping through.
Amount input	`gt=0, max_digits=14, decimal_places=2`	Rejects `1e999999`, NaN and sub-cent values (findings F2, F3).
Unexpected errors	500 with no detail	Only `DomainValidationError` maps to 400 (finding F5).

03

Built with AI, in the open

The context file, agents, skills, hooks, the specification and the commit plan, and how each one shows up as evidence.

The workflow, from context to reviewed PR



- Each step leaves evidence in the repository: rules, specification, code and tests, stronger tests, scores, a browser test, a reviewed PR.
- Agents are read-only wherever they judge work (test-reviewer, pr-reviewer, mutation-analyst); only spec-writer and implementer may write.

Hooks write the log; agents do the work

HOOKS: RUN AUTOMATICALLY ON EVERY CLAUDE CODE EVENT (.CLAUDE/SETTINGS.JSON)

`log_event.py`
Prompt, tool use, sub-agent stop, session stop: one JSON line, 27 fixed keys per event

`check_summary.py`
After shell commands: pytest, mutmut, Playwright become CHECK name | metrics | result=pass/fail

`validate_review.py`
When a sub-agent stops: blocks a pr-reviewer answer that is not the findings JSON schema

→

`logs/ai-interactions.jsonl`
Append-only: prompt, tools, sub-agents, AI suggestion, human decision, verification, status, files

→

`summarize_log.py`
Validates every line and writes the readable summary, counting prompts and human decisions

Secrets, local paths and the account name are removed before a line is written. A hook never fails a session.

```
Human corrections are recorded too: python .claude/hooks/log_event.py --human "what I decided"
```

AGENTS (.CLAUDE/AGENTS) AND SKILLS (.CLAUDE/SKILLS)

`spec-writer`
read, write
writes Spec Kit files only

`implementer`
read, write, shell
one task group at a time

`test-reviewer`
read-only
finds tests that cannot fail

`pr-reviewer`
read-only
answers in findings JSON only

`mutation-analyst`
read-only
survivors to missing tests

Context matters: AGENTS.md

What AGENTS.md holds

The approval rules table with inclusive limits; layering (rules only in `domain/`); the exact commands for tests, mutation and the browser test; review expectations (one verdict per finding); a Never list.

Why a file, not a prompt

It is read at the start of every session and by every sub-agent, so each step starts from the same rules without repeating them.

Same prompt, two agents

Aspect	Without	With AGENTS.md
Boundaries	Whole units: 49,999 / 50,001	One cent: 50,000.00 / .01
Same user	Same name only	Trimmed, case-folded
Admin	Not mentioned	Never approves
Test command	"Find it in the build files"	The real commands
Findings	Fix, decline or ask	Fixed, dismissed or deferred

One run per arm, and AGENTS.md contains part of the answer by design: read it as an illustration, not a benchmark.

Agents and skills, and what each one caught

Role	Access	Job	What it produced here
spec-writer	read, write	Writes Spec Kit files only	Spec with 7 clarifications, plan, research, data model, contract, 25 tasks
implementer	read, write, shell	One task group at a time, tests first	Single-approver limits, then the two-approver rule
test-reviewer	read-only	Finds tests that cannot fail	Weak state tests, 2 duplicates, a fake that hid missing saves, 14 missing cases
mutation-analyst	read-only	Classifies surviving mutants	16 survivors, all missing tests (none equivalent); the tests to add
pr-reviewer	read-only	Answers only in the findings JSON format	11 findings, two reproduced against the running code
Skills	loaded on demand	approval-rules, review-findings, run-checks	One source for the rules, the verdict vocabulary and the exact check commands

Hooks and the interaction log

log_event.py

Runs on prompt, tool use, sub-agent stop and session stop. Appends **one JSON line** with 27 fixed keys. Redacts secrets, local paths and the account name.

check_summary.py

Runs after shell commands. Turns pytest, mutmut and Playwright output into `CHECK name | metrics | result=pass/fail` so every run is comparable.

validate_review.py

Runs when a sub-agent stops. **Blocks** a `pr-reviewer` answer that is not the findings JSON schema.

The 27 keys of every line in logs/ai-interactions.jsonl

```
id, timestamp, session, stage, event, agent, agent_name, agent_type, ai_tool, model, mode, related_ids,
parent_task, context, request, summary, skills_used, subagents_used, mcp_tools_used, tools_used, suggestion,
human_decision, action_verification, limitation, status, artifacts, source
```

A prompt gets an id; every later action carries it in **parent_task**, so each edit, command and check traces back to the request that caused it. **python scripts/summarize_log.py** validates every line and writes the readable summary.

Spec Kit: the specification comes first

1 Constitution

6 principles: rules in the domain, test-first with negative cases, boundaries both sides, refusals explain themselves.

2 Specify

3 user stories with acceptance scenarios and the exact feature text.

3 Clarify

7 questions answered in writing: inclusive limits, first approver, same user, Admin.

4 Plan

Plan, research, data model and the approvals API contract.

5 Tasks

T001 to T025 in 5 phases; tests before code for each story.

What is in the folder

`spec.md` `plan.md` `research.md` `data-model.md` `quickstart.md` `tasks.md`
`contracts/approvals-api.md` `checklists/requirements.md` `.specify/memory/constitution.md`

Honest note

Spec Kit was installed with `specify init`; the spec, plan and tasks follow its templates and were written by the assistant directly rather than by running the `/speckit-*` commands one by one.

Built in small steps: the commit plan

#	Commit	Why it stands alone
1	Port backend from ASP.NET Core to Python/FastAPI (behaviour-identical)	No .NET SDK; keeps the 7 original tests green before any feature
2	Add AGENTS.md rules, skills, sub-agents and AI-interaction logging hooks	The context every later step relies on
3	Add Spec Kit specification, plan and tasks for approval limits	The agreed behaviour, before code
4	Enforce Manager and Director approval limits	Single-approver rules, tests first
5	Require two distinct approvers, second a Director, above 500,000	Approvals list, duplicate-user and second-approver rules
6	Add Playwright test for a refused over-limit approval and approval progress in the UI	The browser-level proof
7	Fix PR review findings	Atomic approvals, amount validation, narrower 400 mapping, strict tests

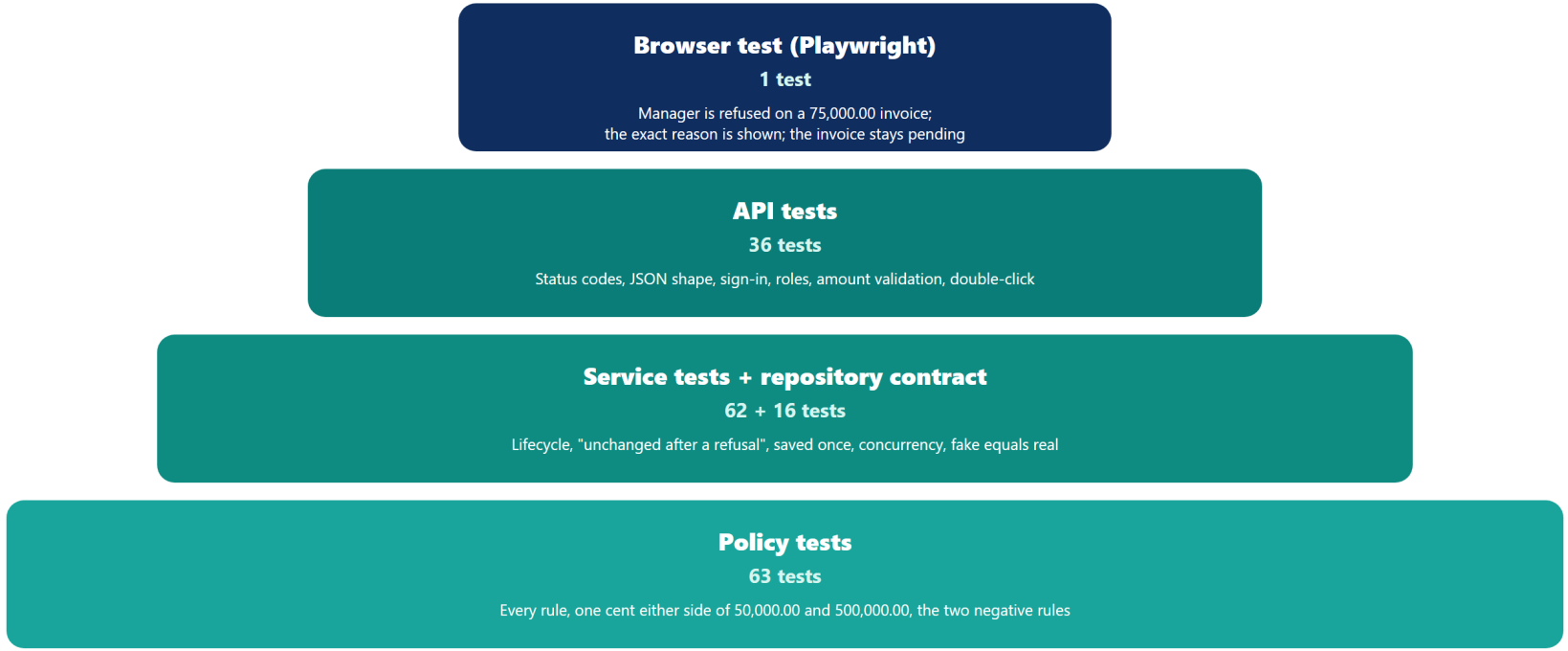
04

Proof that it works

Test layers, the two negative rules, mutation scores, the browser test and the pull-request review.

PROOF

The test layers



Each refusal test asserts the reason text, deep-compares the invoice before and after,
and checks the repository saw no update. A test that only checks the exception type does not count.

Mutation testing

mutmut changes the code on purpose and checks a test fails.

Before: 63.9% (23/36)
After: 100% (132/132)

Hook tests

10 tests in .claude/hooks that prove the logger redacts secrets, keeps 27 keys and survives bad input.

Total

177 backend tests
10 hook tests
1 browser test

PROOF

The two negative rules, tested properly

Same user twice is refused

Including `Dir.One` versus `dir.one`, with the duplicate on the new or the stored side.

A Manager cannot be the second approver

Above 500,000 a Manager may only be first; a Director must give the second approval.

A refusal changes nothing

Each test compares a deep copy before and after, asserts the exact reason text, and checks the repository saw no update.

```
collected 63 items / 51 deselected / 12 selected

backend\tests\test_approval_policy.py::test_second_approver_must_be_a_different_director PASSED                                  [  8%]
backend\tests\test_approval_policy.py::test_second_approver_who_is_a_manager_is_refused_with_a_reason PASSED                     [ 16%]
backend\tests\test_approval_policy.py::test_same_user_cannot_be_both_approvers PASSED                                            [ 25%]
backend\tests\test_approval_policy.py::test_same_user_is_recognised_despite_case_and_whitespace[DIR.ONE] PASSED                  [ 33%]
backend\tests\test_approval_policy.py::test_same_user_is_recognised_despite_case_and_whitespace[ dir.one ] PASSED                [ 41%]
backend\tests\test_approval_policy.py::test_same_user_is_recognised_despite_case_and_whitespace[Dir.One] PASSED                  [ 50%]
backend\tests\test_approval_policy.py::test_same_user_is_recognised_despite_case_and_whitespace[dir.one\t] PASSED               [ 58%]
backend\tests\test_approval_policy.py::test_same_user_check_wins_over_the_role_check PASSED                                     [ 66%]
backend\tests\test_approval_policy.py::test_is_same_user_ignores_case_and_surrounding_whitespace_only PASSED                    [ 75%]
backend\tests\test_approval_policy.py::test_same_user_is_recognised_when_the_stored_name_has_odd_case_or_spacing[ Dir.One ] PASSED [ 83%]
backend\tests\test_approval_policy.py::test_same_user_is_recognised_when_the_stored_name_has_odd_case_or_spacing[DIR.ONE] PASSED [ 91%]
backend\tests\test_approval_policy.py::test_same_user_is_recognised_when_the_stored_name_has_odd_case_or_spacing[dir.one\t] PASSED [100%]

============================================================ 12 passed, 51 deselected in 0.04s ============================================================
```

PROOF

Mutation testing: from 63.9% to 100%

Baseline

ported code, 7 tests: 23 of 36

First pass

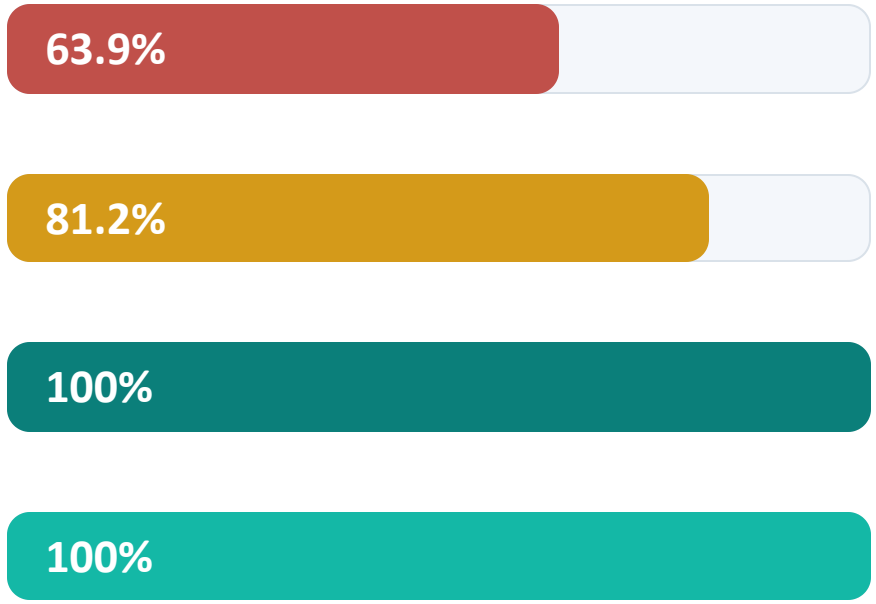
feature + 75 tests: 69 of 85

After

134 tests: 86 of 86

After SOLID refactor

132 of 132



- The mutant count rises from 36 to 132 because the scope grew (new rule files), so compare the **kill rate**, not the counts.
- Mutation testing measured the **domain unit tests** only; the API tests are not in the run.
- A mutant that survives means a change to the code went unnoticed by every test.

Final mutation run

132 / 132

```
Killed: 132   Survived: 0
Timeout: 0   Other: 0
Mutation score: 100.0%
```

- **16 survivors** after the first pass were all missing tests: a fake that never saved, unasserted timestamps and messages.
- The refactor left one survivor (99.2%): a `>` that an earlier rule had been hiding. A boundary test per rule closed it.

PROOF

Browser test: one refusal, end to end

```
cd e2e; npx playwright test --headed; cd ..
```

```
Running 1 test using 1 worker

  ✓  1 …hromium] › tests\over-limit-rejection.spec.ts:14:5 › a manager is refused on an over-limit invoice, sees why, and the invoice does not change (1.4s)

  1 passed (3.1s)

To open last HTML report run:

  npx playwright show-report
```

What the test does

Signs in as `demo.manager`, **creates its own** 75,000.00 invoice, submits it, presses Approve on the Approvals tab, and expects the exact text **A Manager may approve invoices up to 50,000.00; this invoice is 75,000.00 and needs a Director.** The invoice must stay Pending Approval in both the list and the table. It then deletes the invoice, so the server is left as it was.

How it is written

Page objects (`LoginPage`, `InvoicesPage`, `ApprovalsPage`) and role-based locators (`getByRole`, `getByLabel`); no CSS or XPath selectors.

PROOF

Reviewing the PR: 11 findings, each with a verdict

6
fixed

3
dismissed

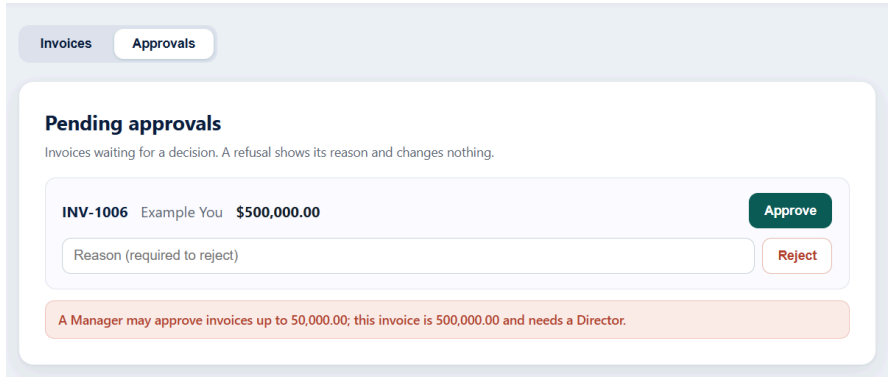
2
deferred

Finding	Verdict	How it was settled
F1 (high) approvals not atomic	fixed	Lock around submit/approve/reject; threaded tests fail without it
F2 (high) `1e999999` breaks the list	fixed	Amount validated at the API; invalid input never stored
F4 `float()` loses precision	dismissed	Input now bounded; every limit round-trips exactly to the cent
F9 roles matched exactly	dismissed	Roles come from the server; failing closed is intended and pinned by a test
F7 no two-Director UI test	deferred	Tracked as T024; the flow is covered at API level
F11 tokens never expire	deferred	Pre-existing demo auth; tracked as T025

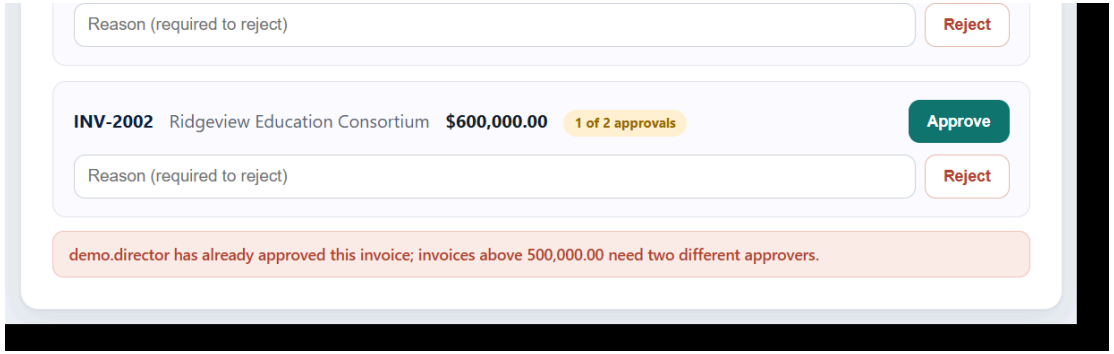
The `test-reviewer` pass raised 14 further gaps in the tests themselves; all were acted on or consciously declined (see the report).

PROOF

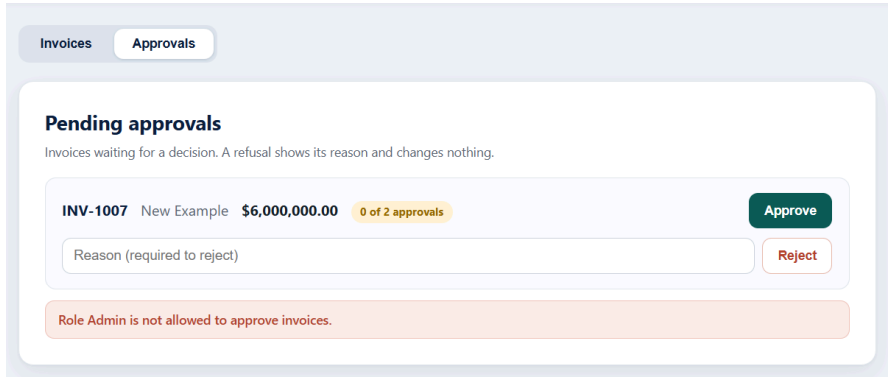
The application in action



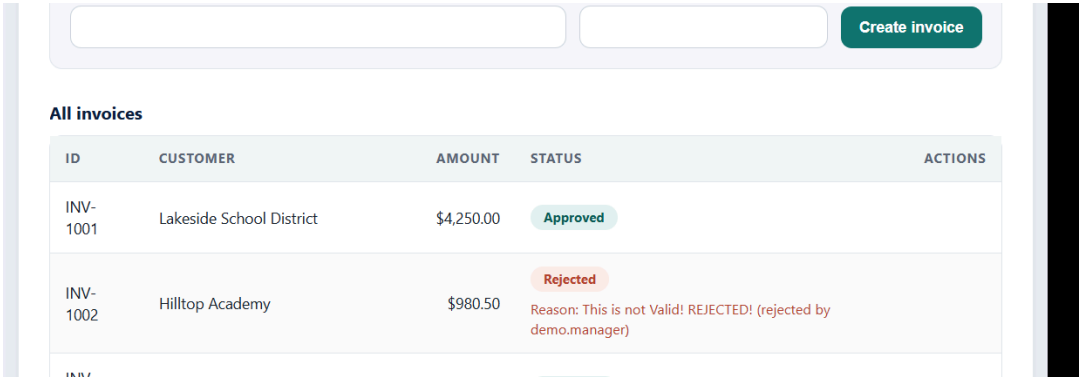
A Manager is refused above 50,000.00, with the reason



The same Director cannot approve twice above 500,000.00



Admin may look, but can never approve



A rejection shows who rejected it and why

05

For the reviewer

Where every hand-in item lives, how to check it yourself in minutes, and what is honestly left open.

FOR THE REVIEWER

Hand-in checklist: where each item is

#	Item	Where to look	Status
1	Pull request, 3 or more standalone commits	Commit plan in `ASSIGNMENT.md` section 5 (7 commits)	Made by the author
2	Updated AGENTS.md	`AGENTS.md`; imported by `CLAUDE.md`	In the repository
3	Spec files	`specs/001-invoice-approval-limits/`, `.specify/`	In the repository
4	Unit tests incl. both negative rules	`backend/tests/` (177 tests)	Passing
5	One Playwright test: rejected over-limit approval	`e2e/tests/over-limit-rejection.spec.ts`	Passing
6	Mutation scores before and after, as text	`ASSIGNMENT.md` section 3; `docs/mutation/*.txt`	63.9% to 100%
7	PR description with a verdict on every finding	`docs/pr-review.md` (11 findings)	Ready to paste
8	Log of all AI interaction	`logs/ai-interactions.jsonl` and its summary	Validated
9	ASSIGNMENT.md with write-up and video link	`ASSIGNMENT.md`	Link added after recording
10	8 to 10 minute demo video	Recorded and linked by the author	Made by the author

FOR THE REVIEWER

Check it yourself in five minutes

1	**Install**	`pip install -e "backend[dev]"`	**once**
2	**Unit tests**	`python -m pytest backend/tests -q`	**177 passed**
3	**Run the app**	`python -m uvicorn invoice_app.api.main:app --app-dir backend/src --port 5000`	**http://localhost:5000**
4	**Browser test**	`cd e2e; npm install; npx playwright install chromium; npm test`	**1 passed**
5	**Mutation**	`wsl -d Ubuntu-22.04 -- bash scripts/run_mutation.sh check`	**100.0% in about 13 s**
6	**AI log**	`python scripts/summarize_log.py`	**validates every line**

Demo accounts (local only): demo.manager, demo.director, demo.director2, demo.admin. Password ChangeMe123!. Try INV-2001 (75,000.00) and INV-2002 (600,000.00).

FOR THE REVIEWER

Honest limits and what is still open

Limits of the evidence

Mutation testing covered the **domain unit tests** only. The AGENTS.md experiment was **one run per arm**. Early planning and the port were logged as `reconstructed`, not captured live. Sub-agents ran as general-purpose agents reading the agent files in the session that created them.

Not production

Sign-in is a **demo**: in-memory sessions, no expiry, no throttling. The store is in memory. Both are documented and outside this feature.

Deferred, with tracking

T024 a second Playwright spec for the two-Director flow, with a reset endpoint so UI tests that change data are repeatable. **T025** session expiry and constant-time password comparison.

Left to the author

Create the pull request from the commit plan, record and link the 8 to 10 minute video, and paste the link into `ASSIGNMENT.md`.

Everything here can be re-run

- Documentation: 23 PDF documents in `docs/pdf/`
- Slides: `docs/presentations/` (this deck and the UI walkthrough)
- Rules of record: `specs/001-invoice-approval-limits/spec.md`
- Evidence: `ASSIGNMENT.md`, `docs/pr-review.md`, `docs/mutation/`, `logs/`